Executive Summary

This assignment implements a compact BERT-style bidirectional Transformer encoder from first principles in PyTorch and applies it to four-way AG News classification. The model is trained from scratch: the notebook defines its own word-level tokenizer, vocabulary, positional encoding, scaled dot-product multi-head self-attention, encoder blocks, padding mask, and [CLS]-based classification head. It does not load pretrained BERT weights or use torch.nn.TransformerEncoder.

Headline result

The validation-selected model used 2 encoder layers, 2 attention heads, model width 128, and learned positional embeddings. On the untouched official test split it achieved 82.26% accuracy and 0.8223 Macro F1 (7,600 examples).

Task objectives

- Classify each combined news title and description into World, Sports, Business, or Sci/Tech.
- Use a balanced training subset of 20,000 examples, with 5,000 examples per category, and a separate 2,000-example validation set.
- Build an additive key-padding mask with 0 for valid keys and -infinity for padding keys; do not apply a causal mask.
- Compare depth, head count, embedding width, and learned versus sinusoidal positional encoding.
- Report training curves, validation comparison, final test classification metrics, a 4 x 4 confusion matrix, and final-layer [CLS] attention visualizations.

Main measured outcomes

Measure	Recorded result
Configurations executed	5 (FAST_EXPERIMENT = True)
Best validation Macro F1	0.8342 - L2 / h2 / d128 / learned
Official test accuracy	0.8226 (82.26%)
Official test Macro F1	0.8223
Best test class F1	Sports: 0.8894
Lowest test class F1	Business: 0.7823

Interpretation boundary: the notebook ran the fast five-configuration comparison, not the complete 16-configuration factorial grid. All five models reached the three-epoch cap without satisfying the specified loss-change convergence threshold.

1. Dataset and Data Preparation

The notebook reads the AG News Kaggle train.csv and test.csv files. Each record is converted into one text sequence by concatenating the cleaned title and description. Numeric labels are mapped from the dataset convention 1-4 to internal class IDs 0-3.

Internal ID	News category	Full training split	Balanced train	Validation	Official test
0	World	30,000	5,000	500	1,900
1	Sports	30,000	5,000	500	1,900
2	Business	30,000	5,000	500	1,900
3	Sci/Tech	30,000	5,000	500	1,900
	Total	120,000	20,000	2,000	7,600

Table 1. Dataset counts shown by the executed notebook outputs.

Split strategy

- The full training CSV contains 120,000 rows, evenly distributed across the four labels (30,000 each).
- A reproducible stratified sample of exactly 5,000 examples per class forms the 20,000-example training subset (seed = 42).
- A further 500 examples per class are selected from the remaining original training rows for validation; these 2,000 rows do not reduce the 20,000 training examples.
- The official test.csv contains 7,600 examples, with 1,900 per class, and is kept separate for final evaluation.

Tokenization and sequence construction

The tokenizer lowercases text and uses a regular expression to retain alphanumeric words, contractions, and punctuation as separate tokens. Vocabulary counts are computed only from the 20,000 training examples. The vocabulary is capped at 15,000 entries, including three reserved tokens:

Special token	ID	Purpose
[PAD]	0	Fills unused positions; masked in attention.
[UNK]	1	Represents tokens not found in the training vocabulary.
[CLS]	2	Prepended summary token; its final hidden state is classified.

Each input starts with [CLS], is truncated to at most 63 text tokens, and is padded to a fixed total length of 64. The resulting array shapes are X_train (20,000 x 64), X_val (2,000 x 64), and X_test (7,600 x 64).

2. System Architecture

The model processes the whole headline-description sequence bidirectionally. Because there is no triangular causal mask, a token can attend to valid tokens on either side of it. Only padding positions are excluded as attention keys.

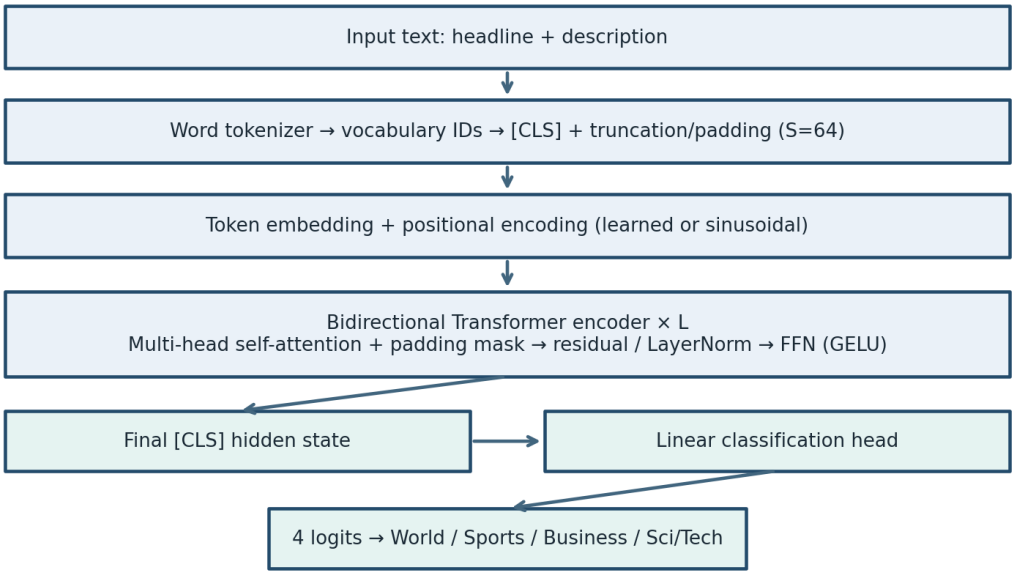


Figure 1. End-to-end processing flow reconstructed from the notebook implementation.

Architecture stages

Stage	Operation	Output / role
1. Input	Concatenate title and description.	Raw news text.
2. Tokenize	Lowercase, split words/contractions/punctuation, map to IDs.	64 token IDs including [CLS] and [PAD].
3. Embed	Token embedding plus position vector.	Sequence representation of width d_model.
4. Encode	Repeat L bidirectional encoder blocks.	Contextual token representations.
5. Pool	Take the final hidden state at position 0 ([CLS]).	One vector per news item.
6. Classify	Dropout followed by a linear layer.	Four class logits.

Best model dimensions

For the selected configuration, L = 2, h = 2, and d_model = 128. The per-head dimension is d_k = d_model / h = 64. The position component is learned. Each encoder layer expands the feed-forward intermediate dimension to 4 x 128 = 512 before projecting back to 128.

3. Model Components and Core Operations

3.1 Positional encoding

Token embeddings represent token identity but not its order. The notebook therefore adds a positional vector to every token embedding: E = Embedding(x) + P_enc. In learned mode, P_enc is a trainable embedding table of shape (sequence length, d_model). In sinusoidal mode, the sine/cosine position matrix is deterministic and registered as a buffer, so it is not updated during training.

3.2 Multi-head self-attention

Each attention block makes separate linear projections for queries (Q), keys (K), and values (V). Each projection is split into h heads. For each head, scaled dot-product attention is computed as:

$$\mathbf{Attention(Q, K, V) = softmax((QK^T / \sqrt{d_k}) + M_pad)\, V}$$

The additive mask M_pad has shape (B, 1, 1, S). It contains 0 at valid key locations and negative infinity at [PAD] key locations. After softmax, padded keys receive zero probability. The attention outputs from all heads are concatenated and passed through an output projection.

3.3 Encoder block

The encoder uses pre-layer normalization and residual additions. The notebook implements the sequence:

$$\mathbf{x \leftarrow x + MHSA(LN(x))}$$

$$\mathbf{x \leftarrow x + FFN(LN(x))}$$

The feed-forward network consists of Linear(d_model, 4d_model), GELU, dropout, and Linear(4d_model, d_model). Dropout is 0.10. The final encoder output passes through LayerNorm, and its [CLS] vector is sent to a dropout-plus-linear classification head producing four logits.

3.4 Training objective

The model is optimized with cross-entropy loss. The training loop uses AdamW, gradient clipping with maximum norm 1.0, and automatic mixed precision when CUDA is available. The notebook execution reports PyTorch 2.11.0+cu130 and a Tesla T4 GPU.

4. Experimental Design

The notebook defines a complete 16-combination grid from L in {2, 4}, h in {2, 4}, d_model in {64, 128}, and positional encoding in {learned, sinusoidal}. The executed setting is FAST_EXPERIMENT = True, so five representative configurations were trained. The baseline is modified one factor at a time for direct controlled comparisons.

Setting	Value	Role / rationale
Random seed	42	Python, NumPy, and PyTorch seeds set for repeatability.
Training subset	20,000	Balanced; 5,000 per class.
Validation set	2,000	500 per class, sampled from remaining train rows.
Vocabulary	15,000	Word-level vocabulary including special tokens.
Maximum sequence length	64	One [CLS] token plus text tokens and padding.
Batch size	128	Used by training and evaluation loaders.
Optimizer	AdamW	Learning rate 3e-4; weight decay 1e-4.
Dropout	0.10	Applied in embeddings, attention/FFN paths, and head.
Epoch cap	3	Fast-mode safety cap.
Convergence tolerance	1e-4	Stop if absolute consecutive train-loss change is below this value.
Device / precision	Tesla T4 / AMP	CUDA available; mixed precision enabled.

Configurations actually trained

Run	Layers (L)	Heads (h)	Width (d)	Position	Comparison
1	2	2	64	Learned	Baseline
2	4	2	64	Learned	Depth increased
3	2	4	64	Learned	Head count increased
4	2	2	128	Learned	Model width increased
5	2	2	64	Sinusoidal	Position scheme changed

Evaluation protocol

The best run was selected using validation Macro F1. The saved code restores that run's best validation checkpoint and evaluates the official test split once. Other configurations therefore have no official test metrics in the results table.

5. Training Behaviour and Epoch Results

Training loss decreased in every configuration during all three epochs. The 128-dimensional learned-position model began with the lowest training loss (1.07259) and ended at 0.54802. The sinusoidal-position run had the highest loss at each epoch (1.35054, 1.04017, 0.83685).

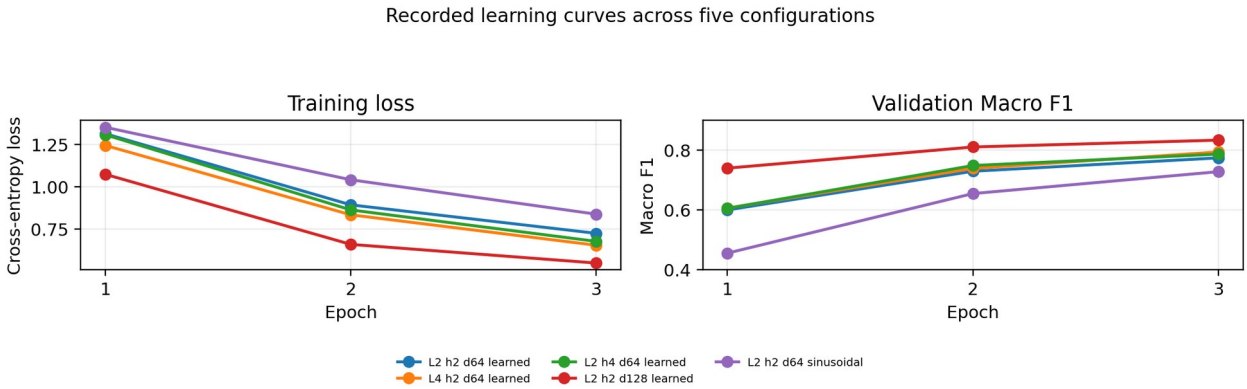


Figure 2. Training-loss curves and validation Macro F1 curves, reproduced from the notebook epoch logs.

Epoch-by-epoch measurements

Configuration	Epoch	Train loss	Val. loss	Val. acc.	Val. Macro F1
L2 h2 d64 learned	1	1.31241	0.97527	0.6045	0.6005
L2 h2 d64 learned	2	0.89194	0.72121	0.7320	0.7301
L2 h2 d64 learned	3	0.72358	0.61911	0.7760	0.7748
L4 h2 d64 learned	1	1.24336	0.95768	0.6125	0.6067
L4 h2 d64 learned	2	0.83313	0.68131	0.7405	0.7395
L4 h2 d64 learned	3	0.65327	0.56856	0.7960	0.7946
L2 h4 d64 learned	1	1.30611	0.96652	0.6110	0.6058
L2 h4 d64 learned	2	0.86196	0.68374	0.7485	0.7492
L2 h4 d64 learned	3	0.67598	0.59019	0.7885	0.7873
L2 h2 d128 learned	1	1.07259	0.66816	0.7425	0.7404
L2 h2 d128 learned	2	0.65825	0.53430	0.8110	0.8113
L2 h2 d128 learned	3	0.54802	0.49412	0.8345	0.8342
L2 h2 d64 sinusoidal	1	1.35054	1.15863	0.4740	0.4559
L2 h2 d64 sinusoidal	2	1.04017	0.83828	0.6630	0.6553
L2 h2 d64 sinusoidal	3	0.83685	0.71414	0.7290	0.7286

The displayed values are the actual per-epoch values recorded by the notebook. Validation accuracy and Macro F1 improved monotonically across epochs for every run.

6. Configuration Comparison and Model Selection

All five models completed three epochs. The convergence flag is False for every run because no adjacent-epoch average training-loss change fell below 1e-4. The final epoch therefore reflects the maximum-epoch cap, not early convergence.

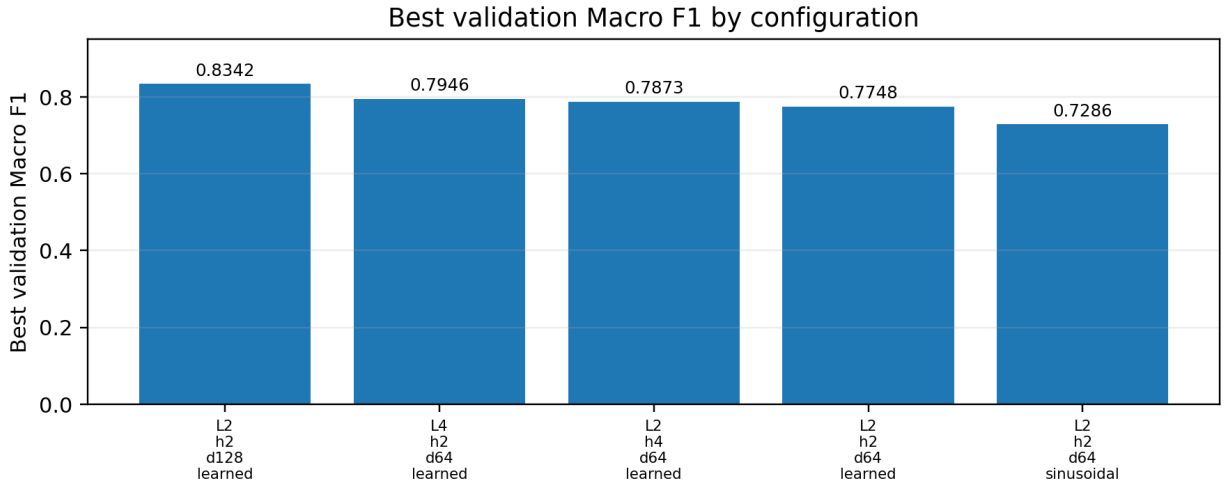


Figure 4. Best validation Macro F1 for each executed configuration (higher is better).

Configuration	L	h	d	Position	Best val F1	Epochs / conv.	Runtime
L2 h2 d128 learned	2	2	128	Learned	0.8342	3 / No	10.6s
L4 h2 d64 learned	4	2	64	Learned	0.7946	3 / No	15.7s
L2 h4 d64 learned	2	4	64	Learned	0.7873	3 / No	10.2s
L2 h2 d64 learned	2	2	64	Learned	0.7748	3 / No	20.7s
L2 h2 d64 sinusoidal	2	2	64	Sinusoidal	0.7286	3 / No	10.3s

What the controlled comparisons show

- Model width: increasing d_model from 64 to 128 (with L=2, h=2, learned positions) produced the strongest result. Validation Macro F1 rose from 0.7748 to 0.8342 (+0.0593 using the unrounded notebook values).
- Depth: increasing layers from 2 to 4 at h=2, d=64, learned positions improved validation Macro F1 from 0.7748 to 0.7946 (+0.0197 using unrounded values).
- Head count: increasing h from 2 to 4 at L=2, d=64, learned positions improved validation Macro F1 from 0.7748 to 0.7873 (+0.0125).
- Position scheme: the learned-position baseline reached 0.7748, compared with 0.7286 for sinusoidal positions at the same L, h, and d (-0.0463 for the sinusoidal run).

Selected checkpoint

L=2, h=2, d_model=128, learned positional embeddings. Its best validation Macro F1 was 0.8342 and validation accuracy after epoch 3 was 0.8345.

7. Official Test Evaluation

After selecting the highest-validation-Macro-F1 checkpoint, the notebook evaluates it on all 7,600 official test records. The model correctly classifies 6,252 examples and misclassifies 1,348. The accuracy is 6,252 / 7,600 = 0.8226.

Test metric	Value	Meaning
Accuracy	0.8226	Fraction of all test items classified correctly.
Macro Precision	0.8226	Unweighted mean of per-class precision.
Macro Recall	0.8226	Unweighted mean of per-class recall.
Macro F1	0.8223	Unweighted mean of per-class F1.
Weighted F1	0.8223	F1 weighted by class support; equal supports here.

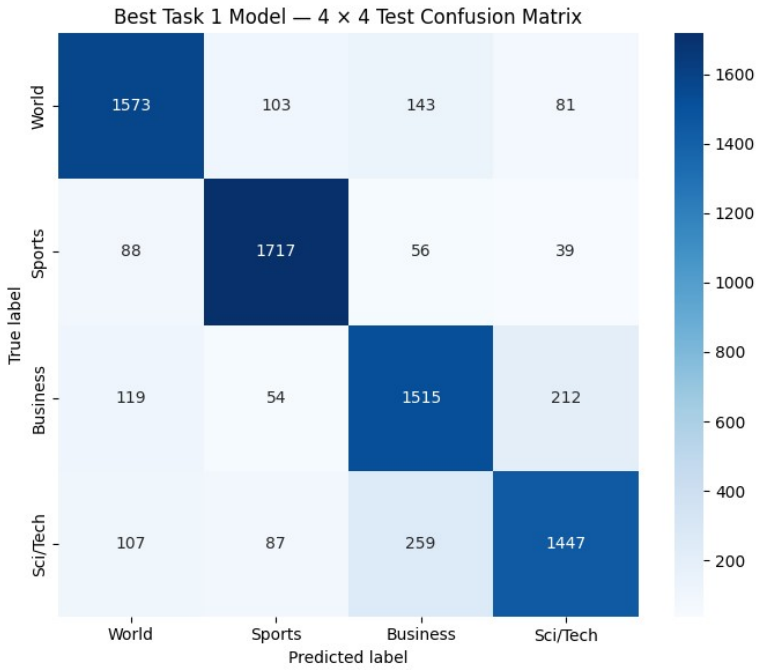
Per-class classification report

Class	Precision	Recall	F1-score	Support
World	0.8336	0.8279	0.8307	1,900
Sports	0.8756	0.9037	0.8894	1,900
Business	0.7679	0.7974	0.7823	1,900
Sci/Tech	0.8134	0.7616	0.7866	1,900
Macro avg	0.8226	0.8226	0.8223	7,600
Weighted avg	0.8226	0.8226	0.8223	7,600

4 x 4 test confusion matrix

Rows are true labels and columns are predicted labels. Diagonal values are correct predictions; off-diagonal values are errors.

True \ Predicted	World	Sports	Business	Sci/Tech
World	1,573	103	143	81
Sports	88	1,717	56	39
Business	119	54	1,515	212
Sci/Tech	107	87	259	1,447



8. Error Analysis and [CLS] Attention

8.1 Confusion patterns

- Sports is the strongest class: 1,717 of 1,900 Sports examples are correctly predicted; recall is 0.9037 and F1 is 0.8894.
- Business has the weakest F1 (0.7823). A notable error is 212 Business examples predicted as Sci/Tech.
- Sci/Tech has 1,447 correct predictions and 259 examples predicted as Business. The Business and Sci/Tech pair is the largest cross-confusion in both directions.
- World is classified correctly 1,573 times; its largest off-diagonal destination is Business (143 examples).

The symmetric pattern of Business/Sci-Tech errors suggests that some stories in these categories share vocabulary or subject matter. This is a plausible interpretation of the observed confusion counts, not a causal conclusion from the model.

8.2 Last-layer [CLS] attention visualization

The notebook visualizes two views for each head in the final encoder layer: attention FROM the [CLS] query to all valid tokens, and attention TO the [CLS] key from every valid token. The example is a Business article; the predicted class is also Business.

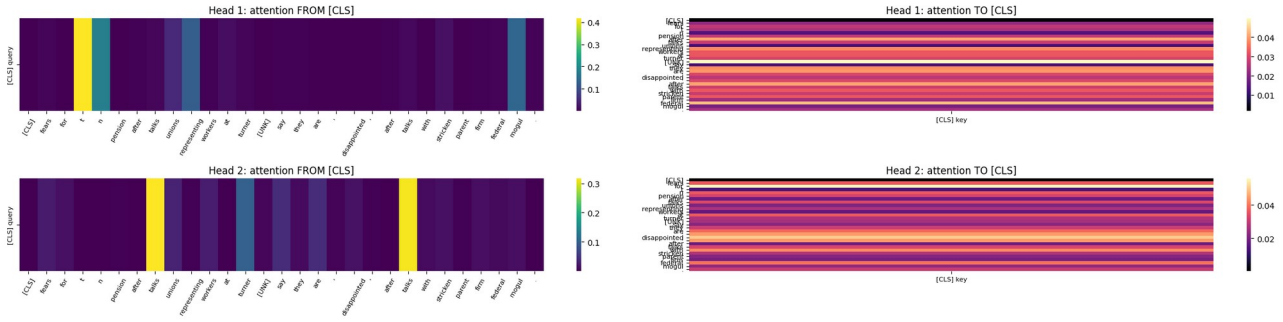


Figure 6. Actual final-layer attention heatmaps for the two heads of the selected model.

The example sequence begins with [CLS] and includes tokens such as “fears”, “pension”, “talks”, “unions”, “workers”, “federal”, and “mogul”. Both attention heads can access tokens across the sequence because the encoder has no causal mask. Their distributions differ, showing that heads assign different weights to the same input. The heatmap is descriptive: attention weights alone should not be treated as a complete explanation of the classifier decision.

9. Discussion, Strengths, and Limitations

What worked well

- The complete encoder pipeline was implemented in ordinary PyTorch modules and tensor operations, including custom multi-head attention rather than a pretrained or built-in Transformer encoder.
- The train/validation split is balanced across all four categories, and the test set remains separate for a final held-out measurement.
- The model uses a padding mask with the intended additive values (0 and -infinity) and no causal mask, consistent with bidirectional attention.
- The width-128 learned-position configuration showed the best validation Macro F1 and a balanced test result, with overall accuracy and Macro F1 both near 0.823.

Important limitations

- Fast-mode coverage: only five representative configurations were executed. Results cannot establish the best configuration across the full 16-combination grid.
- Short training: every run stopped at the three-epoch cap, and every convergence flag is False. The training loss was still falling at epoch 3; a longer run could change the ranking or final test performance.
- Training from scratch: no pretrained BERT weights or transfer learning were used. The architecture is BERT-style in its bidirectional encoder design, but it is not pretrained BERT.
- Word-level tokenization: the vocabulary is limited to 15,000 tokens and unseen words map to [UNK]. It does not use subword segmentation, so rare words and word fragments may be poorly represented.
- Fixed sequence window: each title-description text is truncated/padded to 64 tokens. Information after the first 63 text tokens is not represented.
- Subset size: the model learns from 20,000 training examples rather than the full 120,000-row training split. This makes the run faster but limits the data available to learn.

Implementation and run observations

The saved notebook reports PyTorch 2.11.0+cu130, CUDA availability, and an NVIDIA Tesla T4 GPU. It also emits some DataLoader worker-shutdown AssertionError messages during iterator cleanup. These messages appear alongside completed epoch logs; all five runs, the final test evaluation, and the saved result figures are present. They should be treated as runtime-cleanup noise in this saved execution, not as a substitute for a clean rerun before final submission.

10. Conclusion and Future Work

The assignment demonstrates a working bidirectional Transformer encoder for four-class AG News classification, implemented from first principles in PyTorch. The preprocessing pipeline builds a 15,000-entry word vocabulary, prepends [CLS], pads sequences to length 64, and constructs the additive key-padding mask required for bidirectional attention. The encoder combines learned or sinusoidal positions, multi-head self-attention, pre-layer normalization, residual connections, GELU feed-forward layers, and a [CLS]-based linear classification head.

Of the five executed configurations, the two-layer, two-head, width-128 model with learned positional embeddings produced the best validation Macro F1 (0.8342). On the one-time official test evaluation it achieved 82.26% accuracy and 0.8223 Macro F1. Sports was the strongest class, while Business and Sci/Tech showed the largest mutual confusion.

Recommended next experiments

- Run the complete 16-configuration grid, keeping the same data splits and validation-based model selection.
- Increase the epoch cap to 5-10 and monitor validation Macro F1, loss curves, and the 1e-4 training-loss convergence rule.
- Compare multiple random seeds to estimate run-to-run variability.
- Test subword tokenization and longer sequence lengths to reduce [UNK] usage and retain more article context.
- Add per-class error analysis with representative misclassified headlines to understand Business/Sci-Tech overlap.

References and source material

1. Submitted Jupyter notebook: “Task1_Bidirectional_Transformer_AG_News_FAST (1).ipynb” (code, executed outputs, metrics, loss curve, confusion matrix, and attention heatmap).

2. AG News Classification Dataset, Kaggle. Dataset source link embedded in the submitted notebook: https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset

3. PyTorch was used for model construction and training; scikit-learn was used for stratified sampling and classification metrics as shown in the submitted code.

Final reporting note

This report reflects the recorded fast-mode execution only. It does not claim that all 16 hyperparameter combinations were run, that the model converged, or that the from-scratch encoder is equivalent to a pretrained BERT model.